

# Trees

## Binary Tree vs. Binary Search Tree

A binary tree is a tree data structure in which each node can have at most two children. These children are referred to as the left child and right child. There is no required ordering between the values stored in the nodes. For example, a node containing 50 could have 30 as its left child and 70 as its right child, but another binary tree could have 80 on the left and 20 on the right. The main requirement is simply that each node has no more than two children.

A Binary Search Tree (BST) is a binary tree that follows an additional ordering rule. For every node, values in its left subtree are smaller than the node's value, while values in its right subtree are larger. This ordering makes searching, insertion, and deletion more efficient than in an unordered binary tree when the tree is reasonably balanced.

For example:

```
    50
   /  \
  30   70
 / \   /\
20 40 60 80
```

This is both a binary tree and a BST because every node has at most two children and the values follow the BST ordering rule.

However, this is a binary tree but not a BST:

```
    50
   /  \
  70   30
```

Both nodes are valid children in a general binary tree, but 70 cannot be in the left subtree of 50 and 30 cannot be in the right subtree if we are following the usual BST rule.

In a class-based C++ implementation, a tree is usually made up of Node objects. Each node stores its data and references/pointers to its left and right children.

A basic node class might look like:

```
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};
```

A tree class can then keep a pointer to the first node, known as the root:

```
class BinaryTree {
private:
    Node* root;
};
```

Consider this tree:

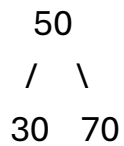
```
    50
   /  \
  30   70
 /  \  /  \
20 40 60 80
```

The following terminology describes the different parts of this structure.

## 1. Node

A node is an individual element of the tree. It contains the data and, in a class-based implementation, pointers to its children.

For example:



50, 30, and 70 are all nodes.

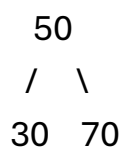
In C++, a node can contain:

```
int data;  
Node* left;  
Node* right;
```

## 2. Root

The root is the topmost node of the tree. It is the starting point from which the rest of the tree can be accessed.

In the example:



50 is the root.

In a class implementation, the root is commonly represented by a pointer:

```
Node* root;
```

If the tree is empty, root is usually nullptr.

### 3. Parent

A parent is a node that has one or more children directly connected below it.

For example:

```
  50
 /  \
30   70
```

50 is the parent of 30 and 70.

Similarly, 30 is the parent of 20 and 40 in the larger tree.

### 4. Child

A child is a node that is directly connected below another node.

For example:

```
  50
 /  \
30   70
```

30 and 70 are children of 50.

A node can have:

No children

One child

Two children

A binary tree never allows a node to have more than two children.

## 5. Left Child

The left child is the node directly connected to the left side of a parent node.

```
    50
   /
  30
```

Here, 30 is the left child of 50.

In C++, it is accessed through:

```
root->left
```

For a BST, the left child and its descendants normally contain values smaller than the parent.

## 6. Right Child

The right child is the node directly connected to the right side of a parent node.

```
50
 \
 70
```

Here, 70 is the right child of 50.

In C++, it is accessed through:

root->right

For a BST, the right child and its descendants normally contain values larger than the parent.

## 7. Leaf

A leaf is a node that has no children.

In the following tree:

```
    50
   /  \
  30   70
 / \   / \
20 40 60 80
```

The nodes:

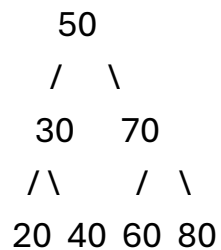
20, 40, 60, 80

are leaf nodes because their left and right pointers are nullptr.

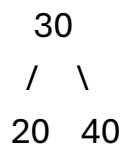
## 8. Subtree

A subtree is a smaller tree formed from a node and all of its descendants.

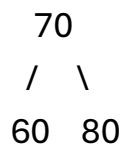
For example:



The subtree rooted at 30 is:



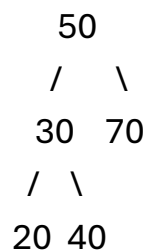
Likewise, the subtree rooted at 70 is:



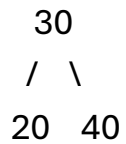
## 9. Left Subtree

The left subtree is the entire subtree that begins with the left child of a node.

For node 50:



The left subtree of 50 is:

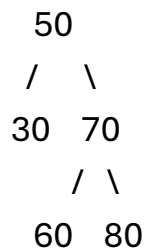


In a BST, all values in this subtree are smaller than 50.

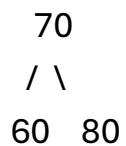
#### 10. Right Subtree

The right subtree is the entire subtree that begins with the right child of a node.

For node 50:



The right subtree of 50 is:

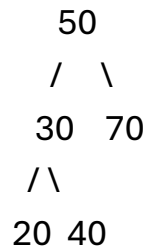


In a BST, all values in this subtree are larger than 50.

## 11. Predecessor

In a BST, the predecessor of a node is the node containing the largest value that is smaller than the current node.

For example:



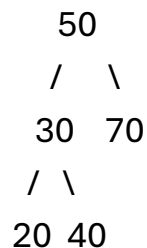
The predecessor of 50 is 40 because it is the largest value smaller than 50.

When a node has a left subtree, its predecessor can normally be found by going to the left child and then moving as far right as possible.

## 12. Successor

The successor of a node is the node containing the smallest value that is larger than the current node.

Using the same tree:



The successor of 50 is 70 because it is the smallest value greater than 50.

When a node has a right subtree, its successor can normally be found by going to the right child and then moving as far left as possible.

## Quick Terminology Summary

Node - An individual element of the tree

Root - The topmost node

Parent - A node that has one or more children

Child - A node directly below a parent

Left Child - Child connected through the parent's left pointer

Right Child - Child connected through the parent's right pointer

Leaf - A node with no children

Subtree - A smaller tree consisting of a node and its descendants

Left Subtree - Subtree beginning at the left child

Right Subtree - Subtree beginning at the right child

Predecessor - Largest value smaller than a given node

Successor - Smallest value larger than a given node

## Binary Search Tree Structure and Terminologies

Node: An individual element of the tree.

Root: The topmost node of the tree.

Parent: A node that has one or more children.

Child: A node directly below a parent.

Left child: Child connected through the left pointer.

Right child: Child connected through the right pointer.

Leaf: A node with no children.

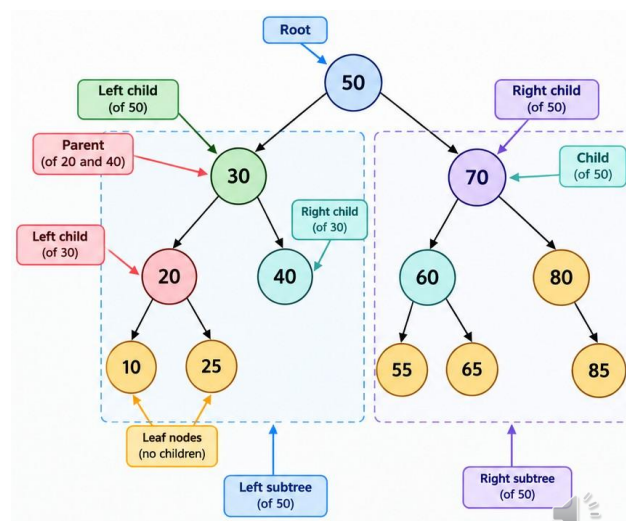
Subtree: A node and all its descendants.

Left subtree: The subtree starting at left child.

Right subtree: The subtree starting at right child.

Predecessor: Largest value smaller than the current node.

Successor: Smallest value larger than the current node.



## Building a Binary Search Tree

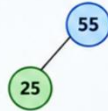
Data Set : 55, 25, 75, 15, 45, 60, 80, 25, 65, 70, 70, 30

### 1 Insert 55



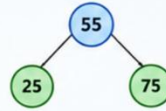
55 becomes the root node.

### 2 Insert 25



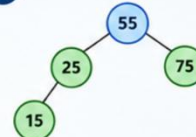
$25 < 55$ , so it goes to the left of 55.

### 3 Insert 75



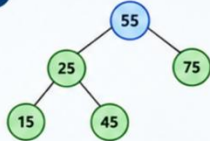
$75 > 55$ , so it goes to the right of 55.

### 4 Insert 15



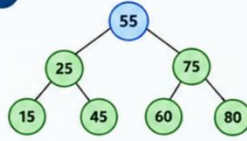
$15 < 25$ , so it goes to the left of 25.

### 5 Insert 45



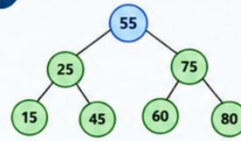
$45 > 25$ , so it goes to the right of 25.

### 6 Insert 60



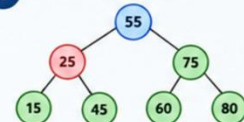
$60 < 75$ , so it goes to the left of 75.

### 7 Insert 80



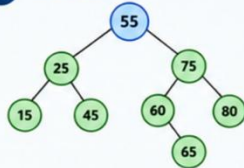
$80 > 75$ , so it goes to the right of 75.

### 8 Insert 25 (duplicate)



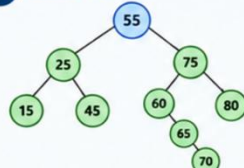
25 is already in the tree.  
Duplicate values are rejected  
(or ignored).

### 9 Insert 65



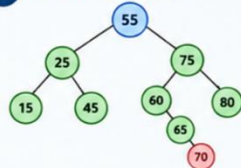
$65 > 60$ , so it goes to the right of 60.

### 10 Insert 70



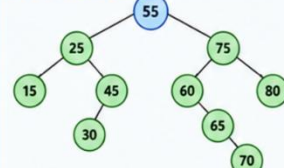
$70 > 65$ , so it goes to the right of 65.

### 11 Insert 70 (duplicate)



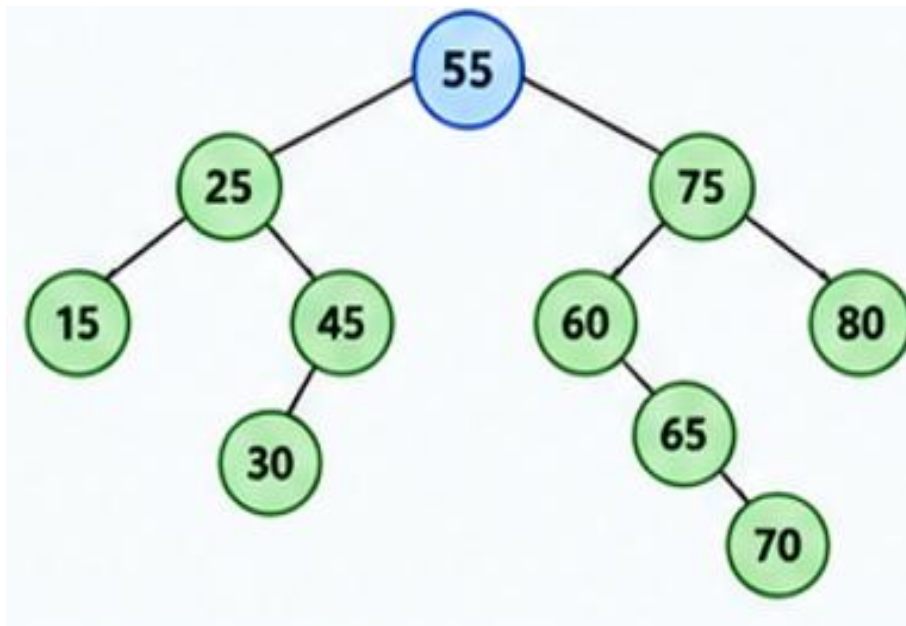
70 is already in the tree.  
Duplicate values are rejected  
(or ignored).

### 12 Insert 30



$30 < 45$ , so it goes to the left of 45.

Final binary search tree is:



Inorder Traversal:

Left → Root → Right

15, 25, 30, 45, 55, 60, 65, 70, 75, 80

Preorder Traversal:

Root → Left → Right

55, 25, 15, 45, 30, 75, 60, 65, 70, 80

Postorder Traversal:

Left → Right → Root

15, 30, 45, 25, 70, 65, 60, 80, 75, 55

Note: Inorder traversal sorts the values.

## C++ Class implementation of Binary Search Tree

```
1  #include <iostream>
2  using namespace std;
3
4  class Node {
5  public:
6      int data;
7      Node* left;
8      Node* right;
9
10     Node(int value) {
11         data = value;
12         left = nullptr;
13         right = nullptr;
14     }
15 };
16
17 class BinarySearchTree {
18 private:
19     Node* root;
20
21     Node* insert(Node* node, int value) {
22         // Create a new node when an empty position is found
23         if (node == nullptr) {
24             return new Node(value);
25         }
26
27         // Insert into the left subtree
28         if (value < node->data) {
29             node->left = insert(node->left, value);
30         }
31
32         // Insert into the right subtree
33         else if (value > node->data) {
34             node->right = insert(node->right, value);
35         }
36
37         // Duplicate value
38         else {
39             cout << "Duplicate value " << value
40                 << " ignored." << endl;
41         }
42         return node;
43     }
44
45     void inorder(Node* node) {
46         if (node == nullptr) {
47             return;
48         }
49
50         inorder(node->left);
51         cout << node->data << " ";
52         inorder(node->right);
53     }
54
55     void preorder(Node* node) {
56         if (node == nullptr) {
57             return;
58         }
59
60         cout << node->data << " ";
61         preorder(node->left);
62         preorder(node->right);
63     }
64
65     void postorder(Node* node) {
66         if (node == nullptr) {
67             return;
68         }
69
70         postorder(node->left);
71         postorder(node->right);
72         cout << node->data << " ";
73     }
74 public:
75     BinarySearchTree() {
76         root = nullptr;
77     }
78
79     void insert(int value) {
80         root = insert(root, value);
81     }
82
83     void inorder() {
84         cout << "Inorder: ";
85         inorder(root);
86         cout << endl;
87     }
88
89     void preorder() {
90         cout << "Preorder: ";
91         preorder(root);
92         cout << endl;
93     }
94
95     void postorder() {
96         cout << "Postorder: ";
97         postorder(root);
98         cout << endl;
99     }
100 };
101
102 int main() {
103     BinarySearchTree tree;
104
105     tree.insert(50);
106     tree.insert(30);
107     tree.insert(70);
108     tree.insert(20);
109     tree.insert(40);
110     tree.insert(60);
111     tree.insert(80);
112
113     // Duplicate test
114     tree.insert(30);
115
116     tree.inorder();
117     tree.preorder();
118     tree.postorder();
119
120     return 0;
121 }
```

```
Duplicate value 30 ignored.  
Inorder: 20 30 40 50 60 70 80  
Preorder: 50 30 20 40 70 60 80  
Postorder: 20 40 30 60 80 70 50
```

## Real World Applications

| Data Structure | Applications                                                         | Why It Is Used                                                                                                                                                                                  |
|----------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stack          | Undo/Redo in applications                                            | Most recent action needs to be undone first.                                                                                                                                                    |
|                | Web browser Back button                                              | Recently visited pages are stored so the most recent is returned.                                                                                                                               |
| Queue          | Print queue                                                          | Print jobs are processed in the order they are submitted.                                                                                                                                       |
|                | CPU/process scheduling                                               | Process waiting for resource are in a queue and handled as they reach the front.                                                                                                                |
| Tree           | File Systems                                                         | Folders can contain subfolders and files, forming a parent-child structure.                                                                                                                     |
|                | HTML/XML documents                                                   | Elements can contain nested elements, creating a hierarchy that can be represented as a tree.                                                                                                   |
| BST            | Searching and maintaining sorted data<br>Symbol tables/ dictionaries | Smaller values on left, larger values on right, allowing comparisons to guide searches.<br><u>Key-value</u> data can be organized by key, allowing efficient searching, insertion and deletion. |

## File Systems as Hierarchical Trees

A file system is naturally represented as a hierarchical tree because folders can contain other folders and files. The root directory acts as the starting point of the hierarchy. Subfolders become child nodes of their parent folders, creating multiple levels of organization. Files can be represented as leaf nodes because they typically do not contain other files or folders.

A folder can contain any number of children, so a file-system tree is not necessarily a Binary Search Tree. Unlike a BST, files and folders do not need to follow a smaller-left/larger-right ordering rule.

This structure makes it easy to navigate from a parent folder to its subfolders and files.

## Example

### Root

